LESSONS — written once, not re-typed every handoff

These do not change with the corpus, so they do not belong in a status print and they do not belong in a dated handoff. Each one cost hours. Referenced by name from the division ledgers.

1. A silent default looks exactly like a working feature

The lead search never ran in a real game across **four** layers of bug. The post-KO search logged every turn and never once decided. Both reported success the entire time.

This is the failure mode this project actually has, and it is invisible to every automated check: a head-to-head, an exploitability search and a prediction score all compare two bots that **share** the blind spot, so a missing capability cancels out exactly.

Make every fallback loud. A default that fires silently is indistinguishable from a feature that works.

2. Search amplifies model error

A maximiser seeks out the lines its model is most optimistic about — which are precisely the lines it is most wrong about. This is why R4 came back negative on the broken engine and positive on the fixed one with the same search and the same flags.

A search is worth exactly what its model is worth. It follows that improving the leaf beats improving the depth, every time, until the leaf is calibrated.

3. Usage counts are sheet counts

Blaze reads 4,585 uses and is worthless: 30 of 54 entries are a Charizard that megas into Drought on turn one, so the ability never fires. Ice Scales, Filter, Aerilate, Prism Armor, Punk Rock and Ripen read zero.

A count of how often something appears on a sheet is not a count of how often it matters.

4. Every derivation over-matches on the first try

- `refusesStatusMoves` caught Telepathy and Wonder Guard
- `speedOnItemLoss` caught Sticky Hold
- `failsIfTargetNotAttacking` caught Quick Guard, Wide Guard and Round

Print what a new tag matched before wiring it. Every time. The list is always longer than expected and the extras are always plausible.

5. My own probes were wrong about fifteen times, always toward a comfortable answer

Three rules that came out of it:

- **Clear the control explicitly.** The first Choice Scarf probe compared a Scarf against a Basculegion that `buildMon` had already given a Scarf, and reported the engine broken.
- **Test the outcome, not the classification.** Whether the code labels something correctly is not whether the thing happens.

- **Identical results across a varied knob mean the knob is unwired.** Not that the knob does not matter.

Two more, added 2026-08-04 after five further cases in one session. The count is now roughly twenty and it is no longer bad luck; it is a property of how probes get written.

- **STAGE IT AGAINST A BODY THAT CAN SHOW THE EFFECT.** Three of the five failed on this alone. The redirection "bug" that led this session's headline for six hours was a probe firing **Dragon** Claw at a **Fairy** type: the redirect worked perfectly, moved the attack onto something immune, and both arms read 0. The `typeImmunity` wire that sat red for two days staged Volt Absorb on a **Garchomp** — Dragon/Ground, already immune to Electric with no ability at all — and *also* left the absorber holding Protect, so it blocked the hit. Two independent reasons that arm could never pass whatever the engine did. And a heal probe on a full-HP body reads `0 → 0` forever. **Ask what the target would do if the mechanic did not exist. If the answer is "the same thing", the probe is worthless.**
- **A FAKE INPUT MUST CARRY THE FIELD THE ENGINE ACTUALLY READS.** A hand-built move object with `cat: 'Physical'` is not a physical move: `medicham2-browser.js:511` is `const phys = mv.c === 'P'`. With `mv.c` absent, `phys` is false for *everything* — which made the sand boost fire where it must not and stopped the snow boost firing where it must, and produced a confident report that **both were broken in opposite directions**. Both were correct. A `mon.moves` entry is a **string**, not an object, so the lookup that was supposed to fetch a real move silently returned nothing and fell through to the fake one. **Prefer a real object from the real path. If you must synthesise one, read the consumer first and match the field names it tests, not the ones that read well.**

Three more, added 2026-08-08 after FIVE cases in a single session. The count is now about twenty-five. Every one of the five accused a mechanic that turned out to be correct, and every one would have cost an engine change that made the engine worse.

- `buildMon` **HANDS YOU THE SPECIES' REAL ABILITY, AND THE OTHER ENGINE'S BODY GETS A DIFFERENT ONE.** A Weather Ball probe read `Meganium into Snorlax` and reported sun at exactly half. The MEDICHAM Snorlax carried **Thick Fat** out of `MC.mons`; the Showdown Snorlax got `abilities[0]` = Immunity. Thick Fat halves Fire and Ice — precisely the arms that went red, because sun is where Weather Ball *becomes* Fire. This is lesson 5's "clear the control explicitly" one rung up: it is not enough to clear the control on the arm under test, **the two engines' bodies must be built to agree, and the probe must assert that they do.** Nothing in this repository asserted it, so every hand-rolled two-engine probe has been one species away from this.
- **AGAINST A NEUTRAL DEFENDER, A WRONG TYPE AND A WRONG MULTIPLIER ARE THE SAME NUMBER.** The same probe offered two hypotheses — "we convert to Water in every weather" and "the BP doubling is missing" — and they fit the data equally because Thick Fat's ×0.5 and a missing ×2 are one factor. A defender chosen for convenience cannot separate them. **Pick the target whose TYPE makes the conversion observable**; the real defect that hunt eventually found (`effMoveType` reading `field.weather` raw while `dmgRange` reads `effWeatherOf`) is invisible on anything but a Ghost, where the correct answer is 115 and ours is 0.
- **AN ARTIFACT IS A PHOTOGRAPH OF THE ENGINE THAT WROTE IT, AND STATUS PRINTS IT FOREVER.** `data/interaction-matrix.json` was two days old, the simulator had moved 141 WIREs past it, and a fix queue was being built off its nineteen parting rows — including a 16,461-use "Shield Dust cluster" the code refutes outright (`:8359` already filters exactly as Showdown does, and WIRE 115 postdates the artifact). The staleness rules were all written about MEASURE and SEARCH artifacts; nobody had applied them to an instrument that measures ENGINE. **Before quoting any**

artifact, compare its mtime to the thing it measured. It is one `stat` call and it would have saved the whole detour.

And the meta-lesson, which is now the loudest signal in this file: across ~25 cases the instrument has been wrong far more often than the engine, and the failure is never random — it always lands on "the engine is broken", never on "my probe is broken", because a probe that finds nothing feels like a wasted probe. **A red result is evidence about the PAIR (probe, engine). Rule out the probe first — it is cheaper to check and it is the more likely culprit.**

6. Will's domain knowledge beat the data repeatedly

Blaze/mega. Gardevoir-is-Trace. Contrary-Staraptor. "Meganium needs the mega." "Reality is not a 1/4 split."

When he says a number looks wrong, check it before defending it.

7. Never read an interim SPRT

66.7% became 44%. 57.7% became 50%. The bound exists exactly so that you do not have to look, and SPRT is only valid under continuous monitoring because its boundaries were derived for it. Reading a Wilson interval repeatedly and stopping when it clears zero does not inherit that property — it badly inflates the false-positive rate.

If you stop a run at a bound, report the SPRT verdict. Not a p-value computed as though n had been fixed in advance.

8. Reuse the canonical path

Hand-rolling a second body builder produced `buildMon("Scizor") -> null`. `dmgMon` already existed and does it right.

The same instinct is why `provenance.js` derives the artifact graph from source instead of carrying a list, and why `status.js` shells out to `provenance.js` rather than reimplementing its staleness rules.

9. Measure the noise floor before believing an effect

Split one arm in half and measure the spread between the halves. An effect smaller than that spread is not an effect, however clean the number looks.

10. Check the corpus stamps before attributing an effect to a lever

SLOWKING's equilibrium and its named rock-paper-scissors cycle justified an entire architecture. The file was computed on the unfiltered store — 87% bots, forfeits and stubs — one day before the quality filter landed. Nothing on the file said so. Re-run on clean data the cycle disappears entirely.

A JSON file on disk looks equally authoritative whether it was generated this morning or before the filter that made it wrong.

11. `git checkout -- <path>` is a DELETE, and it belongs to whoever holds that file

Uncommitted work under that path is gone. There is no reflog for a working-tree overwrite. It has the same permanence as deleting an untracked file, reached through a command that reads like a revert.

2026-08-06. An ENGINE agent had rewritten `tests/test-effective-identity.js` — section 2b, which used to assert that a stone-holder materialises the mega's ability at BUILD time. ROADMAP #31 made that false by design, so the agent rewrote it to assert the contract at BOTH moments: the base forme's ability before the choice, the mega's after a real turn. Uncommitted.

The router, fixing a mangled edit of its own in the same file, ran `git checkout --` on it. That discarded the rewrite and two `DECLARED` entries. The file went back to asserting the chimera and failed 49 megas — **the test was wrong and the engine was right**, which is the direction that gets "fixed" by editing the engine if nobody notices what happened.

Two rules, and the second is the one that generalises.

A file has one holder. That includes the router. The same session had spent the evening enforcing one-writer-per-file on subagents, refusing to touch `engine/game_differential.js` because an agent held it, and then wrote three new files into that agent's tree and reverted a fourth. The rule is not about who is careful; it is about who is in the tree. See CLAUDE.md on a measuring agent beside a writing agent — the invariant is the same one, and the router is not exempt from it.

Ask the author to restore, do not reconstruct. The agent rebuilt it verbatim from its own working context and the assertion counts came back identical — 62/62, 62/62, 61/61 — which is what proves the restoration faithful rather than merely green. A reconstruction by the person who destroyed it would have been a guess at somebody else's design, and it would have passed.

Why this cost twenty minutes and not a day: it went red LOUDLY, against a correct engine. The expensive version is the one where the stale contract lands quietly and goes green again on some future engine that happens to satisfy it.

12. A frozen release freezes the ENGINE and not the READER, and the reader keeps moving

2026-08-12. Will: *"should i be concerned we suddenly cant run old things"*, then *"make sure this never happens again"*.

CORRECTED, AND THE FIRST FIGURES WERE MINE AND WRONG. I reported 168 of 200 releases unopenable and essentially no artifact re-runnable, and told Will to be concerned on that basis. Both numbers came out of a check I had written badly minutes earlier: it UNIONED every caller's `need` list and held every artifact to all 24 symbols, so a release that serves one caller perfectly was reported as stranding that caller's own artifact over a symbol it has never read. Judged per PRODUCING caller, the real state is **29 re-runnable, 1 stranded, 11 unknown-producer** — and "168" is one caller's number wearing the store's name: `game_differential.js` can use 28 of 201 releases where `speed_vs_pokeenv.js` can use 196. The underlying lesson stands and the alarm did not — including `all-mechanics-fire.json`, which carries the 93 move divergences the whole mechanics programme is built on.

Nothing broke that day. The snapshots verify. They hold their bytes. What moved was the HARNESS: every symbol added to a caller's `REL.require(..., {need: [...]})` list retroactively strands every release cut before that symbol existed. The camera is not in the photograph.

THE CAUSE WAS CORRECT WORK, WHICH IS THE WHOLE POINT. 30 snapshots export `rngStreams`, the oldest cut at 2026-08-12T00:19 — ROADMAP #222, splitting the RNG streams so the Protect counter stopped being welded to the accuracy pin. That was the most valuable instrument fix of the week. `spreadL50` stranded two more. Neither should be undone. **The defect is not the change; it is that paying its cost was invisible for a week, and surfaced as eleven scenarios reporting `release THREW` — which reads as eleven broken mechanics and was one unopenable snapshot.**

What the rule is

Do not try to reproduce a stranded artifact. Stop it being citeable.

CLAUDE.md already settles this: *"A quarantined number does not become true when MEDICHAM becomes correct; it becomes RE-RUNNABLE."* Re-running means re-measuring on the current engine, not resurrecting old bytes. So an artifact whose release cannot be opened is not a recovery job — it is a figure that must be WITHHELD rather than annotated, exactly like a quarantined one.

Freezing the reader alongside the engine was considered and REJECTED. It would make old runs reproducible and doubles every snapshot to answer a question nobody asks: nobody wants the number the old harness produced, they want today's number.

What prevents it

1. **A release records what it PROVIDES, at cut time** (`engine/engine_release.js`). Captured when the bytes were known good, so the check is a string comparison instead of parsing 200 snapshots.
2. `tests/test-artifact-rerunnable.js` **ratchets STRANDED-and-undeclared.** A new stranding fails BY NAME. Growing a `need` list then costs a visible, named artifact at the moment it is paid.
3. **The pre-commit hook runs it**, because a check somebody has to remember to run is the thing this file exists to stop. Same reasoning that armed the living-docs hook on 2026-08-06.

An artifact that genuinely should not be re-run declares `"rerun": false` with a reason. That is a person deciding, once, in writing — not a silence.

The part worth remembering when this shape appears again

The failure announced itself as an ENGINE problem. A test went red with eleven scenarios reading `release THREW`, and the natural reading was eleven broken mechanics. It was one aged-out baseline. **When a whole population of checks fails at once, suspect the thing they SHARE before the thing they each test.**